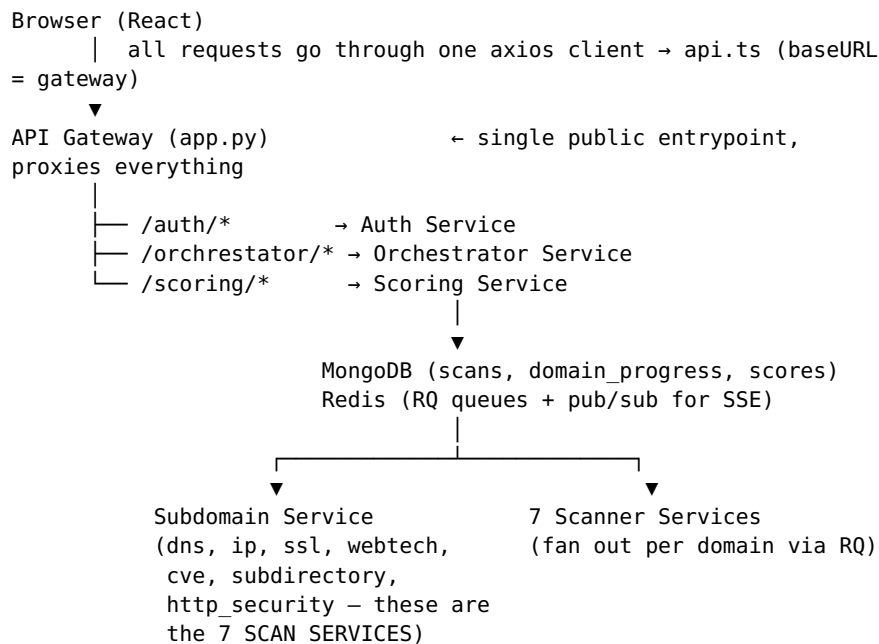


# NTREAT — Frontend ↔ Backend Linkup & Scan Flow

This document explains how the React/TypeScript frontend talks to the Flask microservices backend, how a scan actually runs end-to-end, and how (and whether) subdomain results get pooled into the scan.

---

## 1. High-level architecture



The gateway (app.py) is a thin reverse proxy: every route just calls `proxy_request(SERVICE_URL, path)`, forwards headers/body/cookies, and also sanitizes the domain field in JSON bodies (strips `http(s)://` and trailing slash) before forwarding — so the frontend never has to worry about domain formatting, and neither does any downstream service.

---

## 2. Frontend structure

### Routing (App.tsx)

- `/` → `Home.tsx` — start a new scan, list recent scans.
- `/scan/:domain` → wrapped in `ScandetailsLayout`, which mounts a `ScanProvider` (from `ScanContext.tsx`) around all child routes:
  - `/scan/:domain (index)` → `Overview_page.tsx`
  - `/scan/:domain/score` → `ScoreFactor.tsx`
  - `/scan/:domain/dns, /ip, /ssl, /webtech, /http_security, /cve, /subdomain, /subdirectory` → one page per scanner service.

### Data fetching — centralized in `ScanContext.tsx`

This is the key architectural decision: **only one component fetches data**. All seven service pages plus the overview and score pages just read from a shared React context (`useScanContext`, re-exported as `useScanData` from `Userscandata.tsx`).

ScanProvider: 1. On mount (per :domain param), calls GET /scoring/scan/summary/:domain once for the initial full snapshot. 2. If the scan isn't finished (status !== 'completed'), it opens an EventSource to GET /scoring/scan/stream/:domain (Server-Sent Events). 3. Every SSE message is a **full scan summary** (same shape as the REST endpoint) — not a partial/raw update — so the client can just setSummary(data) wholesale. The stream self-closes once status === 'completed'.

Every page (Dns\_section.tsx, Ipscan.tsx, Ssl.tsx, Web\_tech.tsx, CvePage.tsx, SubdirectoryPage.tsx, SubdomainPage.tsx, HttpSecurity.tsx) follows the same pattern:

```
const { loading, scans } = useScanData();
const scan = scans['<service_name>'];
if (!scan || scan.status !== 'completed' || !scan.results) {
  return <LoadingCard service="..." status={scan?.status ||
'queued'} />;
}
// render scan.results
```

This means no page ever makes its own network call — they're pure consumers of whatever ScanContext currently holds.

### Overview\_page.tsx

Splits services into fastServices = [dns, ip, ssl, webtech] and slowServices = [subdirectory, cve, http\_security] (this mirrors the backend's FAST\_SERVICES / SLOW\_SERVICES constant in the scoring service). While the 4 fast services aren't all done, it shows a "Scanning..." progress screen; once they are, it shows the score, grade, and per-service status grid, calling Overview\_score once every service (fast + slow) is completed.

### Home.tsx

- Starts a new scan: POST /orchrestator/scan with { domain, org\_name }, optionally followed by POST /scoring/domain-name to attach a friendly label, then navigates to /scan/:domain.
  - Lists recent scans via GET /scoring/scans/grouped, polling every 5s while any scan is still active.
  - Deletes a scan via DELETE /scoring/scans/:domain.
- 

## 3. What happens when a scan is started (backend)

1. **Gateway** receives POST /orchrestator/scan → proxies to the **Orchestrator Service** (/scan).
2. Orchestrator creates a scans document (status: "pending") and enqueues process\_orchestration(domain, scan\_id) onto orchestrator\_queue.
3. **process\_orchestration (the orchestration task):**
  - Sets scan status to "scanning\_subdomains".
  - Enqueues process\_subdomain\_scan on subdomain\_queue — this runs the actual subdomain enumeration.
  - **Polls** the scans collection (every SUBDOMAIN\_POLL\_INTERVAL = 2s, up to SUBDOMAIN\_MAX\_WAIT = 600s) for a document with {"scan\_id": scan\_id, "service": "subdomain"} to reach status in ("completed", "failed").
  - If it completes, subdomains are pulled from subdomain\_doc.results.subdomains. If it times out, the orchestrator proceeds with just the root domain and records a

- `subdomain_warning`.
- Builds `all_domains = {root_domain} u discovered_subdomains` (deduped).
- Sets `scans.status = "in_progress"`, `domains = all_domains`, `total_jobs = len(all_domains) * 7`, `completed_jobs = 0`.
- For **each** domain in `all_domains`, inserts a `domain_progress` document (`status: "in_progress"`, one `services.<name> / results.<name>` slot per scanner) and fans out all 7 scanner jobs for that domain via `enqueue_to_all_services(d, scan_id)`.

**So yes — subdomain pooling is done.** The orchestrator explicitly waits for subdomain discovery to finish (or time out) *before* fanning out the 7 scanners, and every discovered subdomain gets its own full 7-service scan and its own `domain_progress` document, exactly like the root domain does.

---

## 4. Completion & scoring flow

- Each of the 7 scanner services writes only its own `services.<name> / results.<name>` path on its domain's `domain_progress` doc (dotted-path \$set, so concurrent scanners never clobber each other), and atomically increments `completed_jobs` as part of that same write (in `orchestrator_client.record_service_result()`).
  - Each scanner then pings `POST /job-done` on the orchestrator with `{scan_id, domain, service}`.
  - `/job-done` calls `check_domain_completion(scan_id, domain)`, which:
    1. Re-reads the domain's current `domain_progress` doc.
    2. If `status != "in_progress"`, it's already been scored — no-op (this guards against duplicate/retried pings).
    3. If `completed_jobs >= total_jobs`, marks the domain "completed" or "partial" (if any service failed), then calls `_score_single_domain()` (writes into the `scores` collection via `scoring_service/logic.py:calculate_domain_score`).
    4. If **every** domain for the scan is now completed/partial, calls `_finalize_scan_score()`, which computes the overall scan score as 50% root domain score + 50% average of subdomain scores (or just the root score if there are no subdomains), and marks the `scans` doc "completed".
  - `sweep.py / sweep_runner.py` is a safety net for missed `/job-done` pings (e.g. orchestrator was briefly unreachable). It runs on a schedule, finds any `domain_progress` doc that's stuck on "in\_progress" but whose counters (`completed_jobs >= total_jobs`) say it's actually done — and only if it's been untouched for at least `STALE_AFTER_SECONDS = 60`, to avoid racing an active scan — then routes it through the exact same `check_domain_completion()` path as `/job-done`, so there's only one place completion/scoring logic lives.
- 

## 5. Live updates to the frontend

- `GET /scoring/scan/summary/:domain` — one-shot, fully assembled snapshot: reads the latest `domain_progress` doc for the domain, per-service status/results, computes `fast_services/slow_services` completion counts, and (once complete) the domain's score — cached in Redis for 1 hour once `status == "completed"`.
  - `GET /scoring/scan/stream/:domain` — SSE endpoint. Subscribes to a Redis pub/sub channel `scan_updates:<domain>`; on every ping it **recomputes the full summary** (not just relays the raw pubsub payload) and streams that down, closing the connection once the domain's overall status is "completed". This is why `ScanContext.tsx` can treat every SSE message identically to the initial REST fetch.
-

## 6. Three-collection MongoDB schema (current)

| Collection      | Keyed by                                            | Purpose                                                                                 |
|-----------------|-----------------------------------------------------|-----------------------------------------------------------------------------------------|
| scans           | scan_id (one doc per whole scan, root_domain field) | Scan-level metadata: status, domains list, job counts, overall score/grade, domain name |
| domain_progress | scan_id + domain (one doc per domain per scan)      | Per-domain per-service status/results, completed/total job counts                       |
| scores          | domain (upserted)                                   | Latest computed per-domain score/grade/breakdown                                        |

Note: scans documents match on root\_domain, **not** domain — routes like /domain-name, /scan/summary, and /scans/:domain (delete) are all careful to query the right field for the right collection.

## 7. Summary

- **Frontend:** one context (ScanContext) owns all fetching (REST + SSE); every service page is a dumb consumer keyed off scans[service].
- **Backend:** gateway proxies → orchestrator drives the scan lifecycle → 7 scanner microservices report back atomically → orchestrator/sweep jointly own “is this domain/scan done yet?” → scoring service computes and caches scores.
- **Subdomain pooling:** ✓ done — the orchestrator waits on subdomain discovery (with a timeout fallback), merges discovered subdomains with the root domain, and scans + scores every one of them individually before rolling them up into one overall scan score.